Análisis Comparativo y Aplicación de Metodologías Ágiles: Scrum, Kanban y eXtreme Programming (XP)

Santiago Andrés Benavides Coral
Código: 20232020036
Andres Felipe Rondón Lara
Código: 20232020005
Davidson Sanchez Gordillo
Código: 20231020183
Facultad de Ingeniería

26 de febrero de 2026

Resumen

El presente informe técnico analiza en profundidad tres de las metodologías ágiles más prevalentes en la ingeniería de software moderna: Scrum, Kanban y eXtreme Programming (XP). Se examinan los orígenes, principios fundamentales, roles, artefactos y ceremonias de cada marco de trabajo. A través de una comparación estructurada, se contrastan sus enfoques respecto a la gestión del flujo, la adaptabilidad al cambio y las prácticas de ingeniería. El documento incluye ejemplos prácticos, diagramas de flujo y una guía de selección para determinar la idoneidad de cada metodología según el contexto del proyecto (incertidumbre de requisitos, madurez del equipo y naturaleza operativa). Se concluye que, si bien Scrum ofrece estructura para la incertidumbre, Kanban optimiza el flujo continuo y XP asegura la excelencia técnica, la elección óptima a menudo reside en la hibridación consciente de estas prácticas.

Índice

1. Introducción **3**

2. Scrum: El Marco de Trabajo Iterativo **3**
2.1. Historia y Principios 3
2.2. Roles, Artefactos y Eventos 3
2.3. Ciclo de Vida (Diagrama) 4
2.4. Ejemplo Práctico 4

3. Kanban: Flujo y Visualización **4**
3.1. Historia y Principios 4
3.2. Tablero y Métricas 5
3.3. Ejemplo Práctico: Soporte IT 5

4. eXtreme Programming (XP) **5**
4.1. Historia y Valores 5
4.2. Prácticas Fundamentales 5

5. Comparación Directa **6**

6. Guía de Selección y Casos de Estudio **6**
6.1. Criterios de Selección 6
6.2. Casos de Aplicación 7

7. Conclusiones y Recomendaciones **7**

1. Introducción

La gestión de proyectos de software ha evolucionado drásticamente desde los modelos tradicionales en cascada hacia enfoques iterativos e incrementales. El *Manifiesto Ágil* (Beck et al. 2001) marcó un hito al priorizar a los individuos e interacciones sobre los procesos y herramientas.

El objetivo de este documento es desglosar las características técnicas y operativas de Scrum, Kanban y XP. Estas metodologías no son mutuamente excluyentes, pero poseen filosofías distintas: Scrum se centra en la gestión empírica de procesos complejos; Kanban, inspirado en el sistema de producción de Toyota, se enfoca en la visualización y optimización del flujo; y XP pone el énfasis en la calidad del código y la retroalimentación técnica inmediata.

2. Scrum: El Marco de Trabajo Iterativo

2.1. Historia y Principios

Scrum fue formalizado por Schwaber y Sutherland (2020) a principios de los 90. Se basa en el empirismo (transparencia, inspección y adaptación) y es ideal para resolver problemas complejos y adaptativos.

2.2. Roles, Artefactos y Eventos

- **Roles:**
  - *Product Owner (PO):* Maximiza el valor del producto y gestiona el Product Backlog.
  - *Scrum Master (SM):* Facilita el proceso, elimina impedimentos y asegura que se entienda Scrum.
  - *Developers:* Equipo autoorganizado que crea el incremento.
- **Artefactos:**
  - *Product Backlog:* Lista ordenada de todo lo que se conoce que es necesario en el producto.
  - *Sprint Backlog:* Elementos seleccionados para el Sprint actual.
  - *Incremento:* La suma de todos los elementos del Backlog completados.
- **Eventos (Ceremonias):**
  - *Sprint Planning:* Definición del objetivo y selección de tareas.

- *Daily Scrum:* Reunión de 15 minutos para sincronización diaria.
- *Sprint Review:* Inspección del incremento con los stakeholders.
- *Sprint Retrospective:* Mejora continua del proceso del equipo.

2.3. Ciclo de Vida (Diagrama)

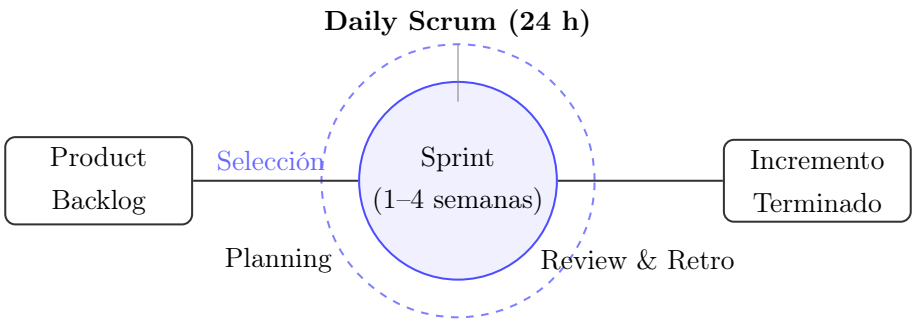

Figura 1: Representación simplificada del ciclo de Scrum.

2.4. Ejemplo Práctico

Una empresa de *fintech* desea lanzar una billetera digital. No tienen claros todos los requisitos de seguridad regulatoria.

1. **Sprint 1:** El PO prioriza el 'Login' y 'Consulta de Saldo'. El equipo se compromete y entrega un incremento funcional en 2 semanas.
2. **Review:** Los stakeholders piden añadir autenticación biométrica.
3. **Adaptación:** El PO añade la biometría al tope del Product Backlog para el siguiente Sprint.

3. Kanban: Flujo y Visualización

3.1. Historia y Principios

Originado en el "Sistema de Producción Toyota", fue adaptado al software por Anderson (2010). No prescribe iteraciones fijas, sino un flujo continuo.

- **Visualizar el trabajo:** Tableros visibles para todos.
- **Limitar el WIP (Work In Progress):** Evitar el cambio de contexto y cuellos de botella.
- **Gestionar el flujo:** Medir y optimizar la velocidad de entrega.

3.2. Tablero y Métricas

Las métricas clave son el *Lead Time* (tiempo desde la petición hasta la entrega) y el *Cycle Time* (tiempo de trabajo activo).

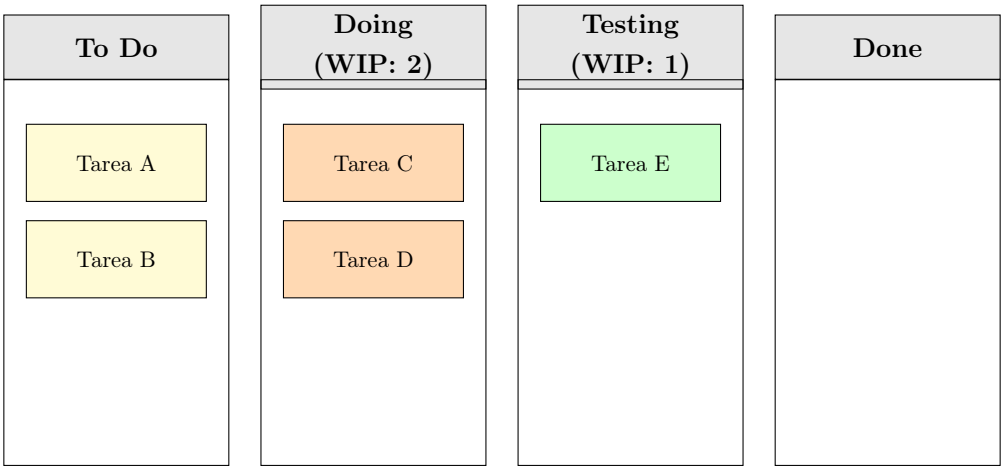

Figura 2: Tablero Kanban con límites de WIP explícitos.

3.3. Ejemplo Práctico: Soporte IT

Un equipo de mantenimiento recibe incidencias aleatorias. Usar Sprints de 2 semanas no tiene sentido. Implementan Kanban:

- Limitan el trabajo en curso a 3 tickets simultáneos.
- Cuando se termina un ticket (Done), se "jala"(*pull*) el siguiente de mayor prioridad.
- Resultado: Reducción del *Lead Time* de 5 días a 2 días.

4. eXtreme Programming (XP)

4.1. Historia y Valores

Creado por Beck (2000), XP lleva las buenas prácticas de ingeniería al extremo. Sus valores son: Comunicación, Simplicidad, Retroalimentación, Coraje y Respeto.

4.2. Prácticas Fundamentales

XP es altamente prescriptivo en el 'cómo' se programa:

1. **Pair Programming:** Dos programadores, una máquina. Mejora la calidad y difunde el conocimiento.

2. **TDD (Test Driven Development):** Escribir la prueba antes que el código.
3. **Refactoring:** Mejora continua del diseño del código sin cambiar su comportamiento.
4. **Integración Continua:** Integrar y probar el código varias veces al día.
5. **Cliente in-situ:** Un representante del cliente trabaja a tiempo completo con el equipo.

5. Comparación Directa

La Tabla 1 resume las diferencias estructurales entre las tres metodologías, apoyada en análisis de Cohn (2009) y Kniberg (2015).

Tabla 1: Comparativa entre Scrum, Kanban y XP

Característica	**Scrum**	**Kanban**	**XP**
Iteraciones	Sprints de longitud fija (1-4 semanas).	Flujo continuo, sin iteraciones obligatorias.	Iteraciones muy cortas (1-2 semanas).
Roles	Prescriptivos (PO, SM, Devs).	No prescribe roles obligatorios.	Prescriptivos (Cliente, Coach, Tracker, etc.).
Cambios	No se permiten cambios en el Sprint actual.	Se permiten si hay capacidad (pull).	Se aceptan cambios si se negocian.
Priorización	Por valor de negocio (Product Backlog).	Por urgencia/valor y clases de servicio.	Estrictamente por el cliente.
Prácticas de Ingeniería	No prescribe (es un marco de gestión).	Opcionales / No prescribe.	**Mandatorias** (TDD, Pair Prog.).

6. Guía de Selección y Casos de Estudio

6.1. Criterios de Selección

- **Usar Scrum si:** El equipo es nuevo en Agile, el proyecto requiere hitos claros, o el feedback del cliente es necesario en intervalos regulares.

- **Usar Kanban si:** El flujo de trabajo es continuo (mantenimiento, soporte), las prioridades cambian diariamente, o se desea visualizar cuellos de botella sin cambiar roles drásticamente.
- **Usar XP si:** El riesgo técnico es alto, los requisitos son vagos pero cambiantes, y se busca la máxima calidad de código posible (M. Poppendieck y T. Poppendieck 2003).

6.2. Casos de Aplicación

Caso A: Startup de Inteligencia Artificial (Entorno Innovador) *Contexto:* Requisitos inciertos, tecnología novedosa. *Solución:* **Scrum + Prácticas de XP**. Se utiliza Scrum para gestionar la incertidumbre del producto mediante Sprints de 2 semanas para validar hipótesis con inversores. Dentro del Sprint, el equipo usa TDD y Pair Programming (XP) para asegurar que el complejo código de IA sea robusto y mantenible.

Caso B: Equipo de DevOps Corporativo (Entorno Operativo) *Contexto:* Peticiones de despliegue, parches de seguridad y errores en producción. *Solución:* **Kanban**. Un Sprint de 2 semanas fallaría porque las urgencias rompen la planificación el primer día. Kanban permite visualizar la cola de tickets, establecer políticas de servicio (SLA) y limitar el trabajo en curso para no saturar al equipo.

7. Conclusiones y Recomendaciones

Ninguna metodología es una "bala de plata". Scrum aporta la disciplina de la gestión temporal y la inspección; Kanban aporta la eficiencia del flujo y la gestión de la capacidad; XP aporta la integridad estructural del software.

Para equipos universitarios o empresas en transición digital, se recomienda:

1. Comenzar con **Scrum** para establecer ritmo y roles claros.
2. Integrar prácticas de **XP** (especialmente TDD y CI) para reducir la deuda técnica.
3. Evolucionar hacia **Kanban** o **Scrumban** si el proyecto entra en una fase de mantenimiento o si el equipo alcanza una madurez alta que no requiere iteraciones forzadas.

Referencias

Anderson, David J. (2010). *Kanban: Successful Evolutionary Change for Your Technology Business*. Sequim, WA: Blue Hole Press.

Beck, Kent (2000). *Extreme Programming Explained: Embrace Change*. Boston: Addison-Wesley Professional.

Beck, Kent et al. (2001). "Manifesto for Agile Software Development". En: *Agile Alliance*. URL: http://agilemanifesto.org.

Cohn, Mike (2009). *Succeeding with Agile: Software Development Using Scrum*. Upper Saddle River, NJ: Addison-Wesley Professional.

Kniberg, Henrik (2015). *Scrum and XP from the Trenches*. 2nd. C4Media.

Poppendieck, Mary y Tom Poppendieck (2003). *Lean Software Development: An Agile Toolkit*. Boston: Addison-Wesley.

Schwaber, Ken y Jeff Sutherland (2020). *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Accedido: 2023-10-27. URL: https://scrumguides.org/scrum-guide.html.